

In the delimiter checker, each element on the stack represents an opening delimiter, that exists, but has not yet been closed. Since it is left hanging, the delimiter checker keeps track of it until it is closed to make sure that by the end of the code all the delimiters are closed. This uses a stack because of the nested way code works. The inside delimiter must be closed before the outside ones do, otherwise it causes errors. For example if { this is opened, then (, when) is pushed, the delimiter checker will check it against the most recent (, so it will match up(), and not against the more previous {.

In the postfix evaluator each element on the stack represents an operand from the stack that has not been applied to an operator. This uses a stack because it uses the 2 most recently pushed operands because the operators act on the operands nearest to them. So, if a list of numbers is pushed, i.e. 8, 7, 3, when the operator comes up, i.e. -, the postfix evaluator will take the 2 most recent, which would be 7 to the left, and 3 to the right, and the result would be $7 - 3 = 4$.